

1. Identificación de Pares: Enumera tres claves (keys) presentes en el nivel superior del objeto JSON.
a) "hostname": "R1-core"; b) "name": "GigabitEthernet0/0" c) "name": "GigabitEthernet0/1",

2. Tipos de Datos: Identifica un ejemplo de cada uno de los siguientes tipos en el código: String, Number, Boolean y Null.

```
{ "hostname": "R1-Core", string
  "uptime_days": 45, int
  "managed_by_apic": true, bool
  "interfaces":
    [ { "name": "GigabitEthernet0/0", string
      "description": "Link to WAN", string
      "enabled": true, bool
      "mtu": 1500 }, int
      { "name": "GigabitEthernet0/1", string
        "description": null, string
        "enabled": false, "mtu": 1500 } ] } int
```

3. { "vlan_id": 10 "name": "Usuarios_Ventas" "tagged": true, "description": "VLAN para el departamento de ventas", } 3. ¿Qué errores de sintaxis impiden que este JSON sea válido según los estándares de red y programación?

C) Existe una coma final (trailing comma) después del valor de 'description'. D) Faltan comas para separar los pares clave-valor después de 'vlan_id' y 'name'.

4. Reflexión Técnica: Explica por qué es crítico que un controlador como Cisco DNA Center reciba un JSON perfectamente validado antes de aplicar cambios en la red.

Es importante que Cisco DNA Center reciba un JSON correctamente validado porque cualquier error en su estructura puede hacer que la información no se interprete de manera correcta. Si se envía un dato equivocado o falta algún elemento, el controlador podría no realizar el cambio esperado en la red. Por eso, antes de aplicar una configuración, es necesario revisar que el JSON esté bien escrito y que todos sus datos sean correctos.

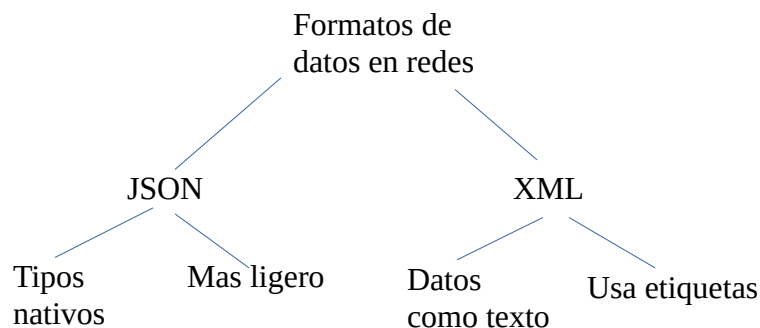
Actividad 2:

1. Escribe el pseudocódigo o completa el script de Python necesario para procesar la variable `inventario_json` (que contiene el texto anterior). El script debe filtrar e imprimir únicamente los hostnames de los dispositivos cuyo estado sea 'down'.

```
inventario = [  
    {"hostname": "Core-SW01", "ip": "10.0.0.1", "status": "up"},  
    {"hostname": "Dist-SW02", "ip": "10.0.0.2", "status": "down"},  
    {"hostname": "Access-SW03", "ip": "10.0.0.3", "status": "up"},  
    {"hostname": "Edge-R01", "ip": "172.16.1.1", "status": "down"}  
]
```

```
##Imprimir todos los hostname  
for h in inventario:  
    print(h["hostname"])  
## imprimir todas las IPs  
for i in inventario:  
    print(i["ip"])  
## Imprimir todos los que esten en down  
for s in inventario:  
    if ["status"] == "down":  
        print(s[status])
```

Utiliza el siguiente organizador gráfico para clasificar las características de JSON y XML basándote en la comparativa anterior y el contenido del Tema 4.



3. ¿Cuál es una diferencia fundamental?

A) JSON tiene tipos de datos nativos (números, booleanos), mientras que en XML todo se trata inicialmente como texto.